24/10/26

CO2 -ASSSESMENT TOOL 2

SCENARIO BASED

NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: N.Umasri **REG.NO: 192424240**

Question 2: Morphological Parsing for Sentiment Analysis

Aim

To develop a Python program to identify prefixes, suffixes, and root words in disagree, agreement, and agreeable, and normalize them for sentiment analysis.

Python Code

```
words = ["disagree", "agreement", "agreeable"]
rules = {
    "disagree": ("dis", "agree", "", "Prefixation",
                 "Opposition or lack of agreement"),
    "agreement": ("", "agree", "ment", "Suffixation",
                  "State or result of agreeing"),
    "agreeable": ("", "agree", "able", "Suffixation",
                  "Pleasant or acceptable")
}
print("MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS")
for word in words:
    prefix, root, suffix, trans, meaning = rules[word]
    print("\nOriginal Word:", word)
    print("Prefix:", prefix or "None")
    print("Root:", root)
    print("Suffix:", suffix or "None")
    print("Transformation:", trans)
    print("Meaning:", meaning)
```

```
print("Normalized Base Form:", root)
```

Output

```
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/co2 at2 NLP.py ==
MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS

Original Word: disagree
Prefix: dis
Root: agree
Suffix: None
Transformation: Prefixation
Meaning: Opposition or lack of agreement
Normalized Base Form: agree

Original Word: agreement
Prefix: None
Root: agree
Suffix: ment
Transformation: Suffixation
Meaning: State or result of agreeing
Normalized Base Form: agree

Original Word: agreeable
Prefix: None
Root: agree
Suffix: able
Transformation: Suffixation
Meaning: Pleasant or acceptable
Normalized Base Form: agree
```

Result

The Python program successfully identifies the prefixes, suffixes, and root words of the given inputs. All three words are normalized to the common root agree. The program also explains their different meanings, helping sentiment analysis systems interpret each word more accurately.

Question 4: Morphological Parsing and Normalization for Document Classification

Aim

To develop a Python program that identifies prefixes, suffixes, and roots in activate, activation, and reactivation, and normalizes them for document classification and semantic indexing.

Python Code

```
words = ["activate", "activation", "reactivation"]

rules = {
    "activate": ("", "act", "ivate",
                 "act -> activate", "Verb",
                 "To make something active"),
    "activation": ("", "act", "ivation",
                   "act -> activate -> activation", "Noun",
                   "Process of making something active"),
    "reactivation": ("re", "act", "ivation",
                     "act -> activate -> reactivate -> reactivation",
                     "Noun", "Process of making active again")
}

print("MORPHOLOGICAL PARSING FOR DOCUMENT CLASSIFICATION")

for word in words:
    prefix, root, suffix, sequence, category, meaning = rules[word]
    print("\nOriginal Word:", word)
    print("Prefix:", prefix or "None")
    print("Root:", root)
    print("Suffix:", suffix)
    print("Derivational Sequence:", sequence)
    print("Word Class:", category)
    print("Meaning:", meaning)
    print("Normalized Base Form:", root)
```

Output

```
======== RESTART: C:/Users/Hedaithulla/OneDrive/Documents/co2 at2 NLP.py =
MORPHOLOGICAL PARSING FOR DOCUMENT CLASSIFICATION

Original Word: activate
Prefix: None
Root: act
Suffix: ivate
Derivational Sequence: act -> activate
Word Class: Verb
Meaning: To make something active
Normalized Base Form: act

Original Word: activation
Prefix: None
Root: act
Suffix: ivation
Derivational Sequence: act -> activate -> activation
Word Class: Noun
Meaning: Process of making something active
Normalized Base Form: act

Original Word: reactivation
Prefix: re
Root: act
Suffix: ivation
Derivational Sequence: act -> activate -> reactivate -> reactivation
Word Class: Noun
Meaning: Process of making active again
Normalized Base Form: act
>
```

Result

The Python program successfully parses the three input words, identifies their morphological components, and normalizes them to the common root act. It demonstrates how suffixes change word class and how the prefix re- adds the meaning “again.” This supports effective document grouping and semantic indexing.